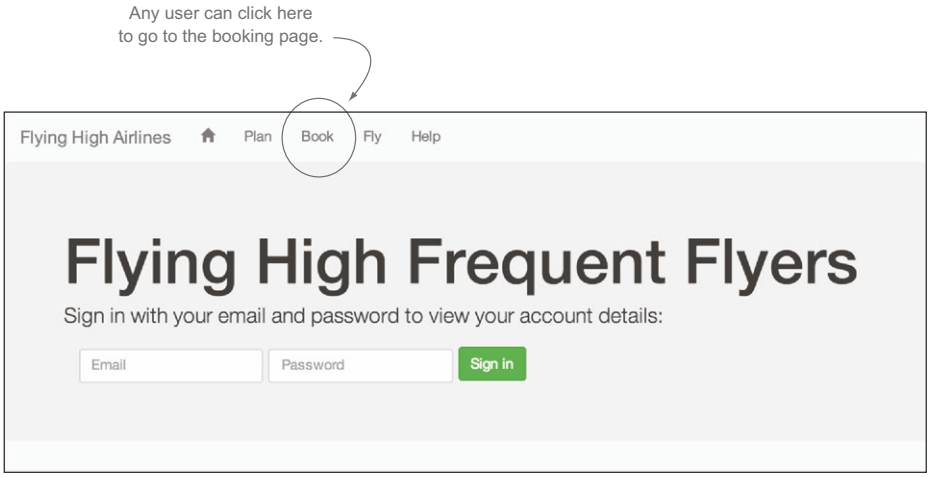


Figure 8.8 Identifying a hyperlink by its text

where other strategies are more convenient. Fortunately, WebDriver provides a number of other ways to identify web elements.

IDENTIFYING ELEMENTS BY LINK TEXT

When you write automated web tests, you often need to click on links, either to navigate to another page or to trigger some action. For example, on the Frequent Flyer site, a user can click on the Book link at the top of the page at any time to go to the booking page (see figure 8.8).

Links like this rarely have a `name` or `id` attribute that you can use to identify them. But you can use the next best thing: the text of the link itself. To click on the Book link, you could write the following:

```
driver.findElement(By.linkText("Book")).click();
```

You can also search the link texts for a partial match. To click on the Flying High Airlines link in the top-left corner of the page, the following call would work:

```
driver.findElement(By.linkText("Flying High")).click();
```

Identifying links by their text content is simple, intuitive, and relatively robust, though the test will obviously break if the displayed text is modified.

IDENTIFYING ELEMENTS USING CSS

A more flexible way of identifying elements is to use *CSS selectors*. CSS selectors are patterns designed to identify different parts of a web page for formatting and styling, but they're also a great general-purpose way to identify elements on the page.

Let's see how CSS selectors can be used for automated web testing. Suppose marketing has asked you to display a list of featured destinations on the home page, as shown in figure 8.9.